RetinAnalysis Setup — Windows

Everything runs in **Anaconda PowerShell Prompt**. No Git Bash needed anywhere in this guide.

> **Skip a step if you already have that tool installed.** Each step says what it's for so you can tell.

1. Install Git

Needed to clone the repository. Download and run the installer, accepting the defaults:

https://git-scm.com/download/win

This installer also bundles Git Bash — you don't need to use it. After installing, `git` itself works directly from Anaconda PowerShell Prompt.

2. Install Docker Desktop

Runs the local MySQL/DataJoint database that stores experiment metadata. Download and install, then launch it once:

https://docs.docker.com/desktop/

Leave Docker Desktop open whenever you need database access (queries, `ra.populate_database()`). Just `import retinanalysis` does not need it running.

3. Install a C++ compiler

One required piece of this package (the `vision-utils` submodule, which reads Vision's binary recording files) compiles C++ extensions the first time it's installed — this needs an actual compiler present, or that install step will fail.

https://visualstudio.microsoft.com/visual-cpp-build-tools/

In the installer, check **"Desktop development with C++"**, then install (a few GB, may prompt a restart). You don't need full Visual Studio or an IDE — the Build Tools package alone is enough.

4. Confirm you have conda

Open **Anaconda PowerShell Prompt** and run:

```
conda --version
```

If that fails, install Miniconda first: https://docs.conda.io/en/latest/miniconda.html

5. Clone the repository

`--recursive` is required — it pulls in a submodule (`vision-utils`) that the package needs to read Vision's data files.

```
git clone https://github.com/yasmine-tani/retinanalysis.git --recursive
cd retinanalysis
```

If you forget `--recursive` , fix it afterward with `git submodule update --init --recursive` .

6. Create the environment and install

```
conda create -y -n retinanalysis python=3.11.13
conda activate retinanalysis
pip install -e .
pip install .\lib\artificial-retina-software-pipeline\utilities\
```

The first `pip install` installs retinanalysis itself and everything it depends on (including `opencv-python` — you never need to install OpenCV separately). The second installs the submodule from step 5 — this is the step that needs the C++ compiler from step 3.

7. Fix a known Windows bug

The first time this package tries to load `matplotlib` , Windows can raise a DLL load error. If you hit that, fix it with:

```
pip uninstall Pillow
pip install -U Pillow
```

If you don't hit the error, skip this — it's not something to run preemptively.

8. Set up config.ini

This file tells the package where your data actually lives on this machine. It's not tracked in git (everyone's paths are different), but a template with the right structure is:

```
cp src\retinanalysis\config\config.ini.example src\retinanalysis\config\config.ini
```

Open `src\retinanalysis\config\config.ini` and edit the `[WINDOWS_DEFAULT]` section with real paths, e.g.:

```
[WINDOWS_DEFAULT]
analysis = B:\Array-data\sorted
data = B:\Array-data\sorted
raw = B:\Array-data\raw
h5 = B:\Array-data\h5
meta = B:\Array-data\dj_meta
tags = B:\Array-data\tags
query = B:\Array-data\sorted
user = your_username
```

`B:` is whatever drive letter your shared network drive is mapped to on this machine — check File Explorer → This PC. `user` is your own username. You only need to fill in `[WINDOWS_DEFAULT]` (and `[WINDOWS_SECONDARY]` only if you actually have a second data location) — leave the other sections alone.

9. Start the local database

Docker Desktop must be open for this to work.

```
mkdir ..\retinanalysis-database
cp docker-compose.yaml ..\retinanalysis-database\
cd ..\retinanalysis-database
docker compose up -d
cd ..\retinanalysis
```

10. Install Jupyter and launch it

```
pip install jupyterlab
jupyter lab
```

This opens a browser tab. Because Jupyter was installed inside the same `retinanalysis` environment you just activated, it's automatically using the right Python — no separate kernel setup needed.

11. Populate the database

Open any notebook in `demos/` and run in a cell:

```
import retinanalysis as ra
ra.populate_database()
```

Quick reference — every command in order

```
git clone https://github.com/yasmine-tani/retinanalysis.git --recursive
cd retinanalysis
conda create -y -n retinanalysis python=3.11.13
conda activate retinanalysis
pip install -e .
pip install .\lib\artificial-retina-software-pipeline\utilities\
cp src\retinanalysis\config\config.ini.example src\retinanalysis\config\config.ini
REM edit config.ini with real paths, then:
mkdir ..\retinanalysis-database
cp docker-compose.yaml ..\retinanalysis-database\
cd ..\retinanalysis-database
docker compose up -d
cd ..\retinanalysis
pip install jupyterlab
jupyter lab
```

Troubleshooting

`ModuleNotFoundError: No module named 'cv2'` on the very first `import retinanalysis` cell

Not a missing OpenCV/C++ install — `opencv-python` is already installed automatically by step 6. This means Jupyter is running a different Python than the `retinanalysis` environment. Close Jupyter, run `conda activate retinanalysis`, then launch `jupyter lab` from that same terminal. Once the notebook is open, check the kernel name in the top-right corner — if it's not `retinanalysis`, switch it (or if it's missing from the list, run `pip install ipykernel` then `python -m ipykernel install --user --name retinanalysis` and refresh).

`DLL load failed` on `import matplotlib`

See step 7.

Installing the `lib\...\utilities\` submodule fails with a compiler error

The C++ Build Tools from step 3 aren't installed, or you need a brand-new PowerShell window opened after installing them.

Database calls fail / connection refused

Docker Desktop isn't open, or the container isn't running. Open Docker Desktop and confirm the `retinanalysis-database` container shows a stop icon (running), not a play icon.

Cloned without `--recursive`

Run `git submodule update --init --recursive`, then redo step 6.